

TABLE OF CONTENTS

Flutter App Architecture: The Repository Pattern	1
What is the repository design pattern?	1
When to use the repository pattern?	3
The repository pattern in practice	3
Parsing the JSON data	5
Initializing repositories in the app	5
Abstract or concrete classes?	6
Using abstract classes	7
Using concrete classes only	7
Writing tests with repositories	7
Mocking the data source	8
Repositories may not need an abstract class	10
The Lowest Common Denominator	10
Repositories scale horizontally	11
Keep it Simple	11
Conclusion	11
Flutter Foundations Course Now Available	12
Flutter Foundations Course	12
Want More?	12
Flutter In Production	13
Flutter & Firebase Masterclass	13
The Complete Dart Developer Guide	13
Flutter Animations Masterclass	13



Flutter App Architecture: The Repository Pattern

#dart

#flutter

#app-architecture

#design-patterns



Andrea Bizzotto

Updated Jan 13, 2022 · 12 min read



Source code on GitHub

Design patterns are useful templates that help us solve common problems in software design.

And when it comes to app architecture, **structural** design patterns can help us decide how the different parts of the app are organized.

In this context, we can use the repository pattern to access **data objects** from various sources, such as a backend API, and make them available as type-safe **entities** to the **domain layer** of the app (which is where our business logic lives).

And in this article, we'll learn about the repository pattern in detail:

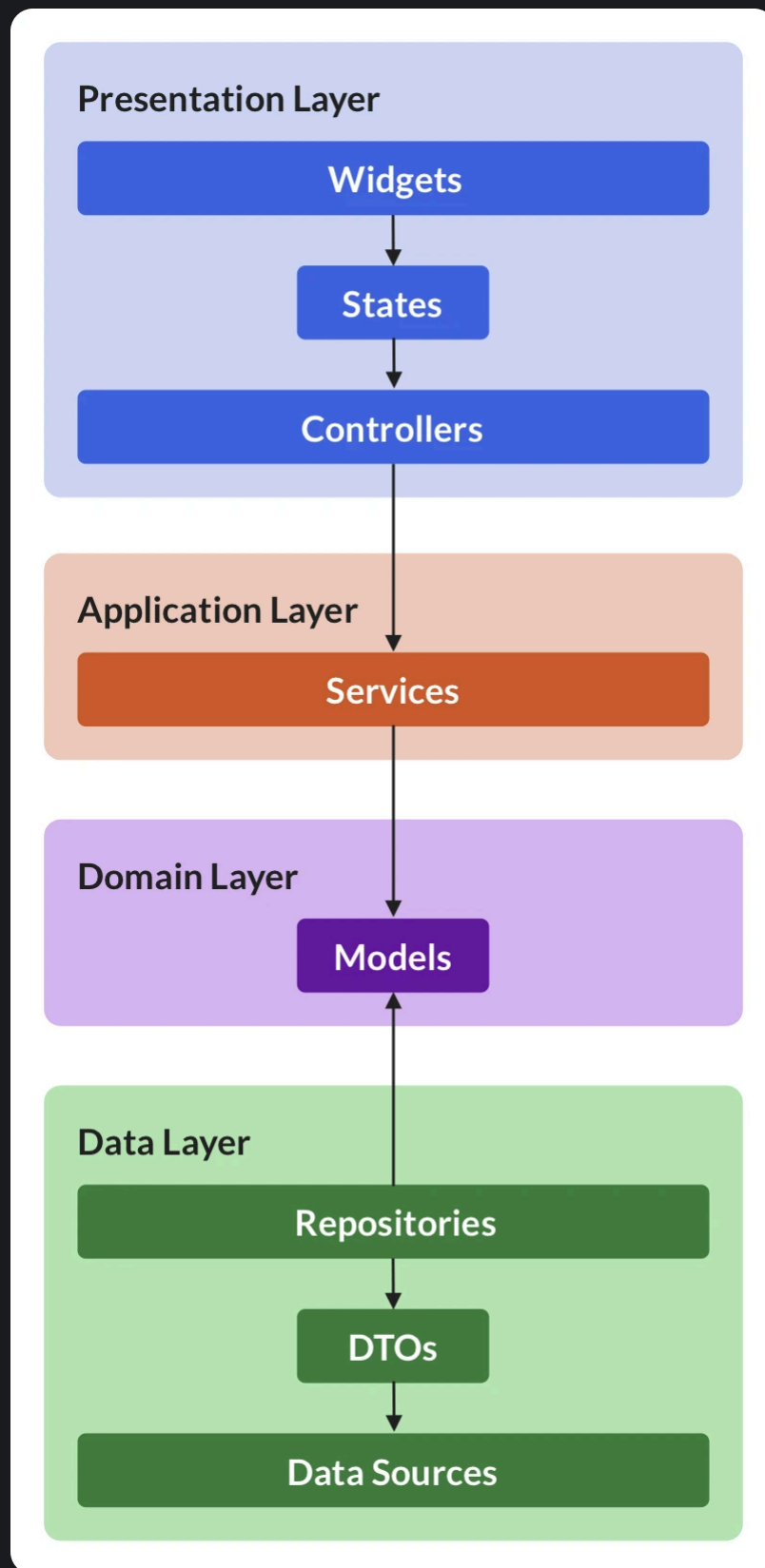
- what it is and when to use it
- some practical examples
- implementation details using **concrete** or **abstract** classes and their tradeoffs
- how to test code with repositories

And I'll also share an example weather app with complete source code.

Ready? Let's dive in!

What is the repository design pattern?

To understand this, let's consider the following architecture diagram:



Flutter App Architecture using controllers, services, and repositories

In this context, repositories are found in the **data layer**. And their job is to:

- isolate domain models (or **entities**) from the implementation details of the data sources in the **data layer**.
- convert **data transfer objects** to validated entities that are understood by the **domain layer**

- (optionally) perform operations such as **data caching**.

The diagram above shows just one of many possible ways of architecting your app. Things will look different if you follow a different architecture such as MVC, MVVM, or Clean Architecture, but the same concepts apply.

Also note how the widgets belong to the **presentation layer**, which has nothing to do with business logic or networking code.

If your widgets work **directly** with key-value pairs from a REST API or a remote database, **you're doing it wrong**. In other words: **do not mix business logic with your UI code**. This will make your code much harder to test, debug, and reason about.

When to use the repository pattern?

The repository pattern is very handy if your app has a **complex data layer** with many different endpoints that return **unstructured data** (such as JSON) that you want to isolate from the rest of the app.

More broadly, here are a few use cases where I feel the repository pattern is most appropriate:

- talking to REST APIs
- talking to local or remote databases (e.g. Sembast, Hive, Firestore, etc.)
- talking to device-specific APIs (e.g. permissions, camera, location, etc.)

One great benefit of this approach is that **if there are breaking changes in any 3rd party APIs you use, you'll only have to update your repository code**.

And that alone makes repositories 100% worth it. *100*

So let's see how to use them! 🚀

The repository pattern in practice

As an example, I've built a simple Flutter app (here's the [source code](#)) that pulls weather data from the [OpenWeatherMap API](#).

By reading the [API docs](#), we can find out how to call the API, along with some examples of response data in JSON format.

And the repository pattern is great for **abstracting away** all the networking and JSON serialization code.

For example, here's an abstract class that defines the **interface** for our repository:

```
abstract class WeatherRepository {  
    Future<Weather> getWeather({required String city});  
}
```

The `WeatherRepository` above has only one method, but there could be more (for example, if you wanted to support all the CRUD operations).

What matters is that the repository allows us to **define a contract** for how to retrieve the weather for a given city.

And we need to **implement** the `WeatherRepository` with a concrete class that makes the necessary API calls using a networking client such as [http](#) or [dio](#):

```
import 'package:http/http.dart' as http;  
  
class HttpWeatherRepository implements WeatherRepository {  
    HttpWeatherRepository({required this.api, required this.client});  
    // custom class defining all the API details  
    final OpenWeatherMapAPI api;  
    // client for making calls to the API  
    final http.Client client;  
  
    // implements the method in the abstract class  
    Future<Weather> getWeather({required String city}) {  
        // TODO: send request, parse response, return Weather object or throw  
        // error  
    }  
}
```

All these implementation details are concerns of the **data layer**, and the rest of the app shouldn't care or even know about them.

Parsing the JSON data

Of course, we'll also have to define a `Weather` model class (or **entity**), along with the JSON serialization code for parsing the API response data:

```
class Weather {  
  // TODO: declare all the properties we need  
  factory Weather.fromJson(Map<String, dynamic> json) {  
    // TODO: parse JSON and return validated Weather object  
  }  
}
```

Note that while the JSON response may contain many different fields, we **only** need to parse the ones that will be used in the UI.

We can write the JSON parsing code by hand or use a code generation package such as [Freezed](#). To learn more about JSON serialization, see my [essential guide about JSON parsing in Dart](#).

Initializing repositories in the app

Once we have defined a repository, we need a way to initialize it and make it accessible to the rest of the app.

The syntax for doing this changes depending on your DI/state management solution of choice.

Here's an example using `get_it`:

```
import 'package:get_it/get_it.dart';  
  
GetIt.instance.registerLazySingleton<WeatherRepository>(  
  () => HttpWeatherRepository(api: OpenWeatherMapAPI(), client:  
    http.Client(),  
  );
```

Here's another using a provider from the [Riverpod](#) package:

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
```

```
final weatherRepositoryProvider = Provider<WeatherRepository>((ref) {  
  return HttpWeatherRepository(api: OpenWeatherMapAPI(), client:  
  http.Client());  
});
```

And here's the equivalent if you're into the flutter_bloc package:

```
import 'package:flutter_bloc/flutter_bloc.dart';  
  
RepositoryProvider<WeatherRepository>(  
  create: (_) => HttpWeatherRepository(api: OpenWeatherMapAPI(), client:  
  http.Client()),  
  child: MyApp(),  
))
```

The bottom line is the same: once you've initialized your repository, you can access it anywhere else in your app (widgets, blocs, controllers, etc.).

Abstract or concrete classes?

One common question when creating repositories is this: **do you really need an abstract class**, or can you just create a concrete class and do away with all the ceremony?

This is a very valid concern since adding more and more methods across two classes can become quite tedious:

```
abstract class WeatherRepository {  
  Future<Weather> getWeather({required String city});  
  Future<Forecast> getHourlyForecast({required String city});  
  Future<Forecast> getDailyForecast({required String city});  
  // and so on  
}  
  
class HttpWeatherRepository implements WeatherRepository {  
  HttpWeatherRepository({required this.api, required this.client});  
  // custom class defining all the API details  
  final OpenWeatherMapAPI api;  
  // client for making calls to the API  
  final http.Client client;
```

```
Future<Weather> getWeather({required String city}) { ... }
Future<Forecast> getHourlyForecast({required String city}) { ... }
Future<Forecast> getDailyForecast({required String city}) { ... }
// and so on
}
```

As is often the case in software design, the answer is: **it depends**.

So let's take look at some pros and cons of each approach.

Using abstract classes

- **Pro:** it's nice to see the interface of our repository in one place, without all the clutter.
- **Pro:** we can swap the repository with a completely different implementation (e.g. `DioWeatherRepository` rather than `HttpWeatherRepository`), and change just one line in the initialization code, because the rest of the app only knows about `WeatherRepository`.
- **Con:** VSCode will get a bit confused when we "jump to reference" and take us to the method definition in the abstract class, rather than the implementation in the concrete class.
- **Con:** More boilerplate code.

Using concrete classes only

- **Pro:** Less boilerplate code.
- **Pro:** "jump to reference" just works as the repository methods will be found in one class only.
- **Con:** swapping to a different implementation requires more changes if we change the repository name (though it's easy to rename things across the entire project with VSCode).

When deciding which approach to use, we should also figure out how to write tests for our code.

Writing tests with repositories

One common requirement during testing is to swap out the networking code with a mock or "fake" so that our tests run faster and more reliably.



However, abstract classes don't give us any advantage here, because in Dart **all classes have an implicit interface**.

This means that we can do this:

```
// note: in Dart we can always implement a concrete class
class FakeWeatherRepository implements HttpWeatherRepository {

  // just a fake implementation that returns a value immediately
  Future<Weather> getWeather({required String city}) {
    return Future.value(Weather(...));
  }
}
```

In other words, there's **no need to create abstract classes** if we intend to mock our repositories in the tests.

In fact, packages like mocktail use this to their advantage and we can use them like so:

```
import 'package:mocktail/mocktail.dart';

class MockWeatherRepository extends Mock implements HttpWeatherRepository {}

final mockWeatherRepository = MockWeatherRepository();
when(() => mockWeatherRepository.getWeather('London'))
  .thenAnswer((_) => Future.value(Weather(...)));
```

Mocking the data source

As you write your tests, you can mock your repositories and return canned responses like we did above.

But there's another option, and that is to **mock the underlying data source**.

Let's recall how the `HttpWeatherRepository` was defined:

```
import 'package:http/http.dart' as http;

class HttpWeatherRepository implements WeatherRepository {
  HttpWeatherRepository({required this.api, required this.client});
```

```
// custom class defining all the API details
final OpenWeatherMapAPI api;
// client for making calls to the API
final http.Client client;

// implements the method in the abstract class
Future<Weather> getWeather({required String city}) {
  // TODO: send request, parse response, return Weather object or throw
  error
}
}
```

In this case, we can choose to mock the `http.Client` object that is passed to the `HttpWeatherRepository` constructor. Here's an example test showing of how you may do this:

```
import 'package:http/http.dart' as http;
import 'package:mocktail/mocktail.dart';

class MockHttpClient extends Mock implements http.Client {}

void main() {
  test('repository with mocked http client', () async {
    // setup
    final mockHttpClient = MockHttpClient();
    final api = OpenWeatherMapAPI();
    final weatherRepository =
      HttpWeatherRepository(api: api, client: mockHttpClient);
    when(() => mockHttpClient.get(api.weather('London')))
      .thenAnswer((_) => Future.value(/* some valid http.Response */));
    // run
    final weather = await weatherRepository.getWeather(city: 'London');
    // verify
    expect(weather, Weather(...));
  });
}
```

In the end, you can choose if you want to **mock the repository itself** or the **underlying data source**, depending on what you're trying to test.

Having figured out how to test repositories, let's get back to our initial question about abstract classes.

Repositories may not need an abstract class

In general, creating an abstract class makes sense if you need many implementations that conform to the same interface.

For example, both `StatelessWidget` and `StatefulWidget` are **abstract classes** in the Flutter SDK, because **they are meant to be subclassed**.

But when working with repositories, you'll likely need only one implementation for a given repository.

Chances are that **you'll only need one implementation for a given repository**, and you can define that as a single, concrete class.

The Lowest Common Denominator

Putting everything behind an interface can also lock you into choosing the **lowest common denominator** between APIs that have different capabilities.

Maybe one API or backend supports **realtime updates**, which can be modeled with a **Stream-based API**.

But if you're using pure REST (without websockets), you can only send a request and get a **single** response, which is best modeled with a **Future-based API**.

Dealing with this is quite easy: just use a stream-based API, and just return a stream with one value if you're using REST.

But sometimes there are broader API differences.

For example, Firestore supports transactions and batched writes. These kinds of APIs use the **builder pattern** under the hood, in a way that is not easily abstracted away behind a generic interface.

And if you migrate to a different backend, chances are that the new API will be considerably different. In other words, **future-proofing your current APIs is often impractical and counter-productive**.

Repositories scale horizontally

As your application grows, you may find yourself adding more and more methods to a given repository.

This is likely to happen if your backend has a **large API surface**, or if your app connects to many different data sources.

In this scenario, consider creating multiple repositories, keeping related methods together. For example, if you're building an eCommerce app, you could have separate repositories for product listings, shopping cart, orders management, authentication, checkout, etc.

Keep it Simple

As usual, keeping things simple is always a good idea. So don't get too wound up overthinking your APIs.

You can model your repository's interface after the API that you need to use, and call it a day. You can always refactor later if needed. 👍

Conclusion

If there's one thing I'd like you to take away from this article, it would be this:

Use the repository pattern to hide away all the implementation details (e.g. JSON serialization) of your data layer. As a result, the rest of your app (domain and presentation layer) can deal directly with **type-safe** model classes/entities. And your codebase will also become more resilient to breaking changes in packages you depend on.

If anything, I hope this overview has encouraged you to think more clearly about app architecture and the importance of having separate **presentation**, **application**, **domain**, and **data** layers, with clear boundaries.

And for more details about this architecture and each individual layer, check the remaining articles in this series:

- [Flutter App Architecture with Riverpod: An Introduction](#)
- [Flutter Project Structure: Feature-first or Layer-first?](#)
- [Flutter App Architecture: The Domain Model](#)
- [Flutter App Architecture: The Application Layer](#)

- Flutter App Architecture: The Presentation Layer

Flutter Foundations Course Now Available

I launched a brand new course that covers Flutter app architecture in great depth, along with other important topics like state management, navigation & routing, testing, and much more:



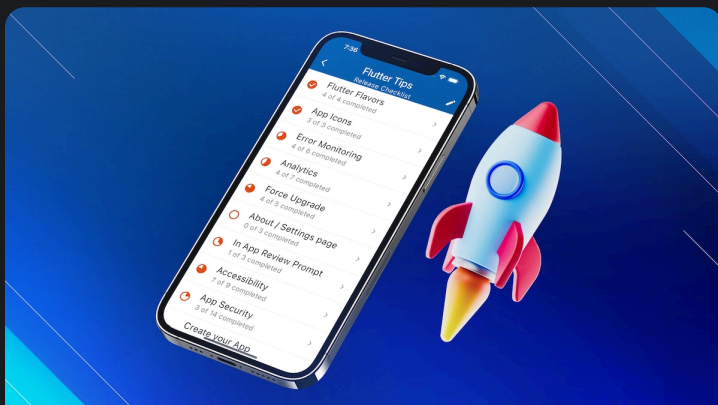
Flutter Foundations Course

INTERMEDIATE TO ADVANCED

Learn about State Management, App Architecture, Navigation, Testing, and much more by building a Flutter eCommerce app on iOS, Android, and web.

Want More?

Invest in yourself with my high-quality Flutter courses.



Flutter In Production

INTERMEDIATE TO ADVANCED

Learn about flavors, environments, error monitoring, analytics, release management, CI/CD, and finally ship your Flutter apps to the stores. 🚀

Flutter & Firebase Masterclass

INTERMEDIATE TO ADVANCED

Learn about Firebase Auth, Cloud Firestore, Cloud Functions, Stripe payments, and much more by building a full-stack eCommerce app with Flutter & Firebase



The Complete Dart Developer Guide

BEGINNER

Learn Dart Programming in depth. Includes: basic to advanced topics, exercises, and projects. Last updated to Dart 2.15.



Flutter Animations Masterclass

INTERMEDIATE

Master Flutter animations and build a completely custom habit tracking application.



CODE WITH ANDREA

Copyright © 2020-present Coding With Flutter Limited

[Contact](#)

[Twitter](#)

[Discord](#)

[GitHub](#)

[RSS](#)

[Meta](#)

[Privacy Policy](#)

[Terms of Use](#)